

Tech Challenge — Fase 2

Sistema Integrado de Oficina Mecânica — Documento de Entrega

Stack: NestJS · TypeScript · Prisma · PostgreSQL · Docker · Kubernetes (kind) · Terraform · JWT · React/Vite

Execução 100% local — sem provisionamento em nuvem

1. Repositório GitHub

Repositório: <https://github.com/engineercodeprojects/fase02-mecanica>

Compartilhamento: repositório compartilhado com o usuário `soat-architecture` (colaborador com acesso de leitura).

Documentação de arquitetura no repo: `docs/arquitetura/arquitetura-local.md` e `docs/arquitetura/arquitetura-fase2.md`.

2. Vídeo de apresentação (≤ 15 min)

Link do vídeo: _____
(YouTube/Vimeo — substituir antes da entrega)

Roteiro sugerido: subida com `docker compose up -d`, consumo das APIs via Swagger, fluxo da Ordem de Serviço, notificação por webhook, e deploy/HPA no cluster `kind local`.

3. Desenho da arquitetura

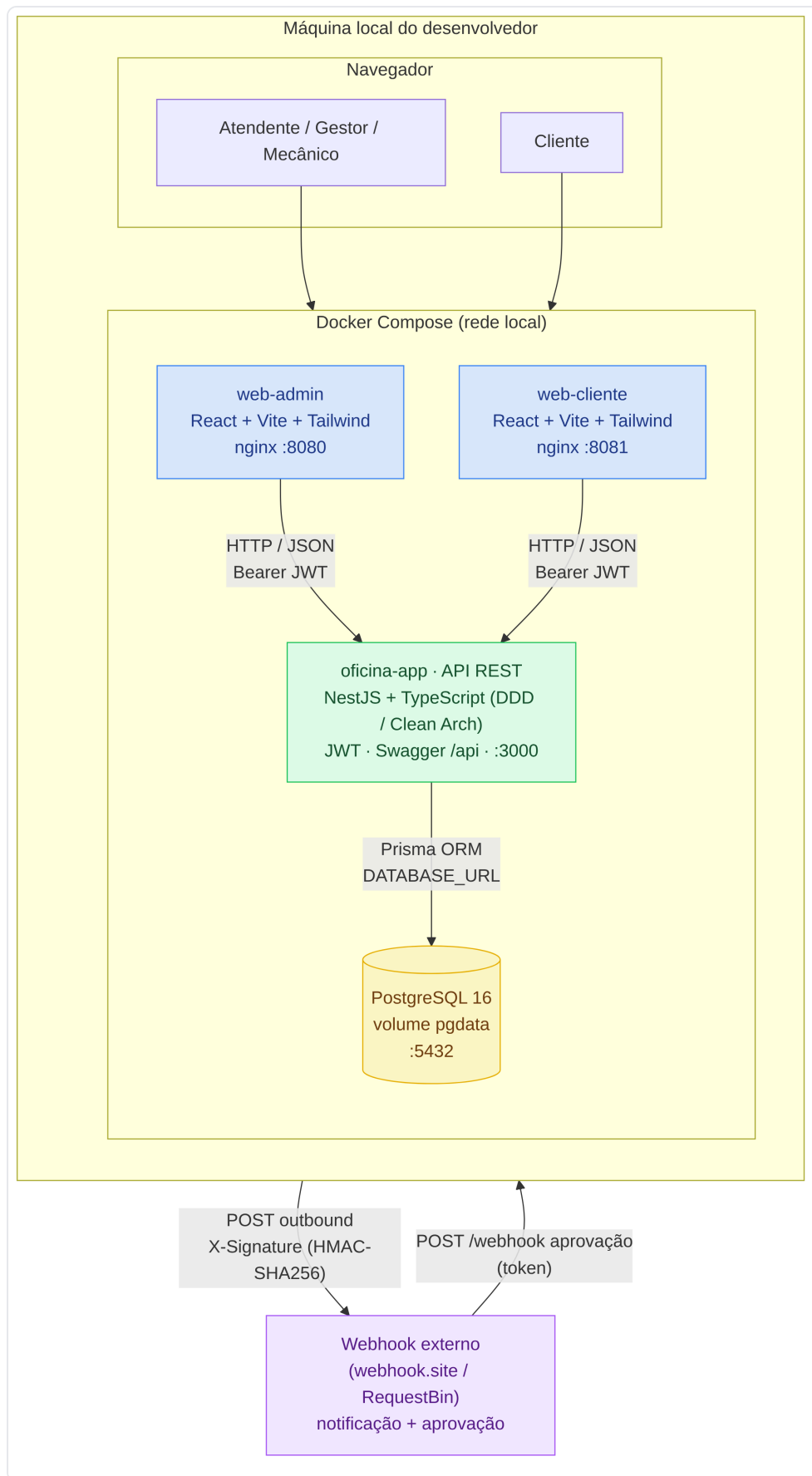
A solução foi desenvolvida e executada **100% localmente**. Os componentes rodam como containers na máquina do desenvolvedor, orquestrados por **Docker Compose** (runtime principal) e, opcionalmente, em um **cluster Kubernetes local (kind)** provisionado por Terraform. O único ator externo é um webhook de terceiros usado para demonstrar notificações e aprovação de orçamento.

3.1. Recursos escolhidos

Componente	Tecnologia	Porta local	Papel
Front Admin	React 18 + Vite + Tailwind (nginx)	8080	Painel para atendente/gestor/mecânico
Front Cliente	React 18 + Vite + Tailwind (nginx)	8081	Acompanhamento e aprovação pelo cliente
API	NestJS + TypeScript (DDD/Clean Architecture)	3000	Regras de negócio, REST, JWT, Swagger
Banco de dados	PostgreSQL 16 (volume <code>pgdata</code>)	5432	Persistência (ACID) via Prisma ORM
Orquestração	Docker Compose	—	Sobe os serviços e aplica migrations

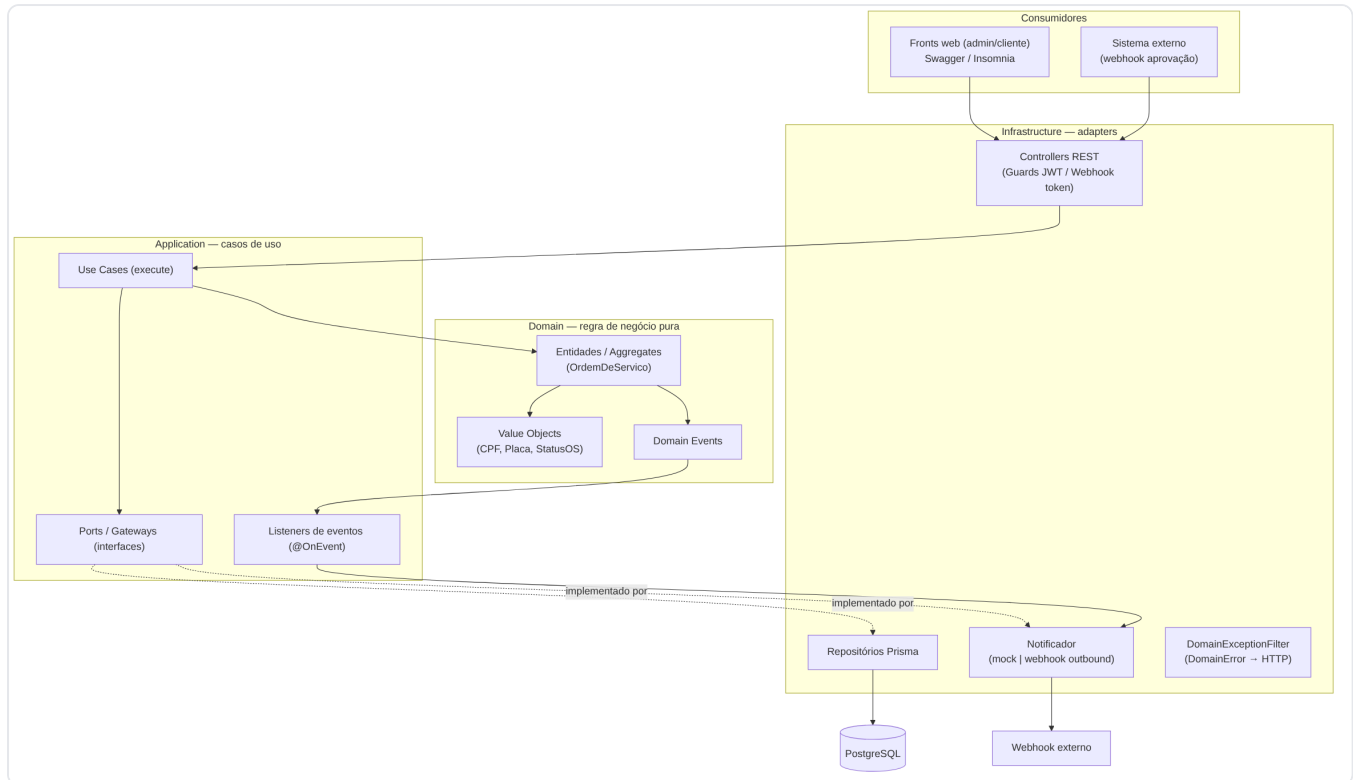
Orquestração (opcional)	Kubernetes <code>kind</code> + Terraform	—	Deploy local com HPA (autoescala)
Integração externa	Webhook HTTP (HMAC-SHA256)	—	Notificação de status + aprovação de orçamento

3.2. Visão geral — runtime local (Docker Compose)



Fluxo: navegador → fronts (nginx) → API NestJS → PostgreSQL; webhook externo para notificação/aprovação.

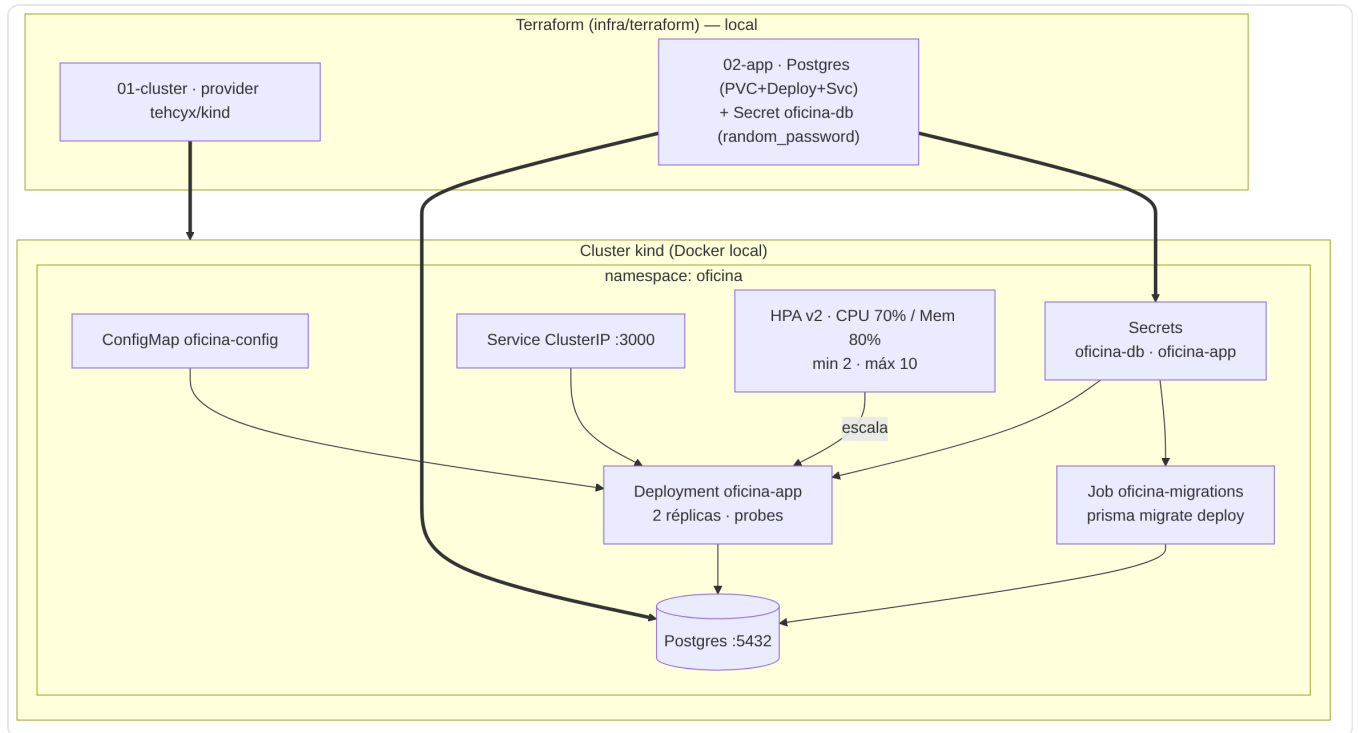
3.3. Componentes da aplicação (Clean Architecture + DDD)



Camadas Infrastructure → Application → Domain, com dependências apontando para o domínio.

Bounded contexts: Atendimento (Cliente, Veículo, OrdemDeServico — aggregate root), Catálogo (Serviço), Estoque (Produto), Autenticação (Usuário/JWT) e Notificação.

3.4. Kubernetes local (opcional) — kind + Terraform



Cluster kind provisionado por Terraform (2 estágios). Sem EKS/RDS — 100% local. HPA min 2 / máx 10.

CI/CD ([.github/workflows/ci-cd.yml](#)) sobe um cluster `kind` efêmero no runner e prova o deploy ponta-a-ponta (testes → build → imagem → terraform apply → kubectl apply → smoke test).